

UNIVERSIDAD PÚBLICA DE EL ALTO

Carrera de Ingeniería de Sistemas

Práctica Integradora

Sistema de Solución de Problemas Reales mediante Teoría de Grafos

Aplicación de:

Construcción de grafos, búsqueda en profundidad,
búsqueda en amplitud, algoritmo de Dijkstra,
algoritmo de Kruskal y algoritmo de Prim

Asignatura: Investigación Operativa II / Estructuras Discretas

Docente: M.Sc. Ing. Juan Carlos Catunta Choquecalles

Gestión 2026

Índice

1. Título de la práctica	3
2. Modalidad de trabajo	3
3. Situación problemática	3
4. Ejemplos de problemáticas	3
5. Objetivo general	4
6. Objetivos específicos	4
7. Conocimientos que se deben integrar	5
8. Parte I: investigación de la problemática	5
8.1. Actividades	5
8.2. Evidencias obligatorias	6
9. Parte II: construcción del grafo	6
9.1. Definición formal	6
9.2. Requerimientos del modelo	7
9.3. Representación obligatoria	7
10. Parte III: aplicación de algoritmos	8
10.1. Búsqueda en profundidad: DFS	8
10.2. Búsqueda en amplitud: BFS	8
10.3. Algoritmo de Dijkstra	8
10.4. Algoritmo de Kruskal	9
10.5. Algoritmo de Prim	9
10.6. Comparación entre Kruskal y Prim	10
11. Parte IV: desarrollo del sistema funcional	10
11.1. Nombre sugerido	10
11.2. Tipo de aplicación	10

11.3. Tecnologías sugeridas	11
12. Funcionalidades mínimas	11
13. Entradas y salidas del sistema	12
13.1. Entradas mínimas	12
13.2. Salidas mínimas	12
14. Casos de prueba obligatorios	13
15. Validación manual	14
16. Evidencias del proceso de desarrollo	14
17. Uso responsable de inteligencia artificial	15
18. Estructura sugerida del proyecto	16
19. Formato sugerido de los datos	16
19.1. Archivo de vértices	17
19.2. Archivo de aristas	17
20. Producto final esperado	17
21. Estructura del informe	18
22. Defensa y demostración	19
23. Preguntas orientadoras para el análisis	19
24. Conclusión esperada	20
25. Criterios de aprobación	20
26. Referencias bibliográficas sugeridas	21

1. Título de la práctica

Diseño e implementación de un sistema funcional para resolver una problemática real mediante teoría de grafos.

2. Modalidad de trabajo

La práctica podrá desarrollarse individualmente o en equipos de un máximo de diez estudiantes, de acuerdo con las indicaciones del docente.

Cada equipo deberá identificar una problemática real, recopilar información verificable, modelarla mediante un grafo e implementar un sistema funcional que utilice los algoritmos estudiados.

No se aceptarán proyectos que únicamente reproduzcan ejemplos genéricos obtenidos de Internet o generados completamente mediante herramientas de inteligencia artificial, sin investigación, datos y evidencias propias.

3. Situación problemática

En diferentes organizaciones y comunidades existen problemas relacionados con rutas, conexiones, distribución de recursos, comunicación, transporte, accesibilidad, infraestructura y planificación de redes.

Estos problemas pueden representarse mediante grafos, donde:

- Los **vértices o nodos** representan lugares, personas, equipos, instituciones, estaciones, aulas, almacenes u otros elementos.
- Las **aristas** representan carreteras, calles, conexiones, relaciones, enlaces o posibilidades de desplazamiento.
- Los **pesos** pueden representar distancia, tiempo, costo, consumo de combustible, riesgo u otra medida cuantificable.

El equipo deberá encontrar una problemática concreta de su entorno y desarrollar un sistema que contribuya a analizarla o resolverla mediante teoría de grafos.

4. Ejemplos de problemáticas

Los siguientes ejemplos son únicamente orientativos. Cada equipo debe contextualizar y justificar su propio problema.

1. Determinación de rutas óptimas para la distribución de productos.
2. Diseño de una red de conexión entre laboratorios o salas de computación.
3. Optimización de recorridos de transporte universitario.
4. Identificación de rutas de evacuación dentro de una institución.
5. Conexión de centros de salud de una determinada zona.
6. Diseño de una red de fibra óptica con costo mínimo.
7. Organización de rutas para la recolección de residuos.
8. Análisis de accesibilidad entre comunidades rurales.
9. Distribución de medicamentos o insumos médicos.
10. Diseño de recorridos para visitas técnicas o mantenimiento.
11. Análisis de conexiones entre dependencias de una empresa.
12. Localización de caminos alternativos ante el cierre de una ruta.

La propuesta elegida deberá contar con una fuente real de información y con una persona, institución, comunidad o grupo que pueda beneficiarse de la solución.

5. Objetivo general

Diseñar e implementar una aplicación funcional que modele una problemática real mediante grafos y aplique los algoritmos DFS, BFS, Dijkstra, Kruskal y Prim para generar resultados útiles en la toma de decisiones.

6. Objetivos específicos

1. Identificar y documentar una problemática real susceptible de ser representada mediante un grafo.
2. Construir el grafo correspondiente, definiendo correctamente vértices, aristas, pesos y tipo de grafo.
3. Implementar la búsqueda en profundidad y la búsqueda en amplitud.
4. Determinar rutas de costo mínimo mediante el algoritmo de Dijkstra.
5. Obtener árboles de expansión mínima mediante los algoritmos de Kruskal y Prim.

6. Comparar los resultados obtenidos mediante los diferentes algoritmos.
7. Desarrollar una aplicación web, de escritorio o móvil que permita interactuar con el grafo.
8. Validar el funcionamiento del sistema mediante datos reales y casos de prueba.
9. Formular recomendaciones relacionadas con la problemática estudiada.

7. Conocimientos que se deben integrar

El proyecto deberá demostrar la aplicación articulada de los siguientes contenidos:

1. Conceptos fundamentales de teoría de grafos.
2. Grafos dirigidos y no dirigidos.
3. Grafos ponderados y no ponderados.
4. Representación mediante listas o matrices de adyacencia.
5. Grado de un vértice.
6. Caminos, ciclos y conectividad.
7. Búsqueda en profundidad (DFS).
8. Búsqueda en amplitud (BFS).
9. Algoritmo de Dijkstra.
10. Algoritmo de Kruskal.
11. Algoritmo de Prim.

8. Parte I: investigación de la problemática

Antes de programar, el equipo deberá realizar una investigación breve del problema.

8.1. Actividades

1. Identificar una problemática del entorno.
2. Describir el lugar, institución o población involucrada.
3. Identificar a los posibles usuarios o beneficiarios.

4. Explicar cómo se resuelve actualmente el problema.
5. Determinar las dificultades o limitaciones existentes.
6. Recopilar datos necesarios para construir el grafo.
7. Definir qué decisiones podría mejorar el sistema propuesto.

8.2. Evidencias obligatorias

El equipo deberá presentar, como mínimo:

- Una entrevista breve, encuesta, observación directa o conversación documentada con un usuario relacionado con el problema.
- Fotografías propias, capturas, croquis, registros, formularios o documentos del contexto investigado.
- Una tabla con los datos originales recopilados.
- Fecha, lugar y responsable de la recopilación.
- Explicación del procedimiento seguido para obtener los datos.

Cuando no sea posible tomar fotografías por razones de privacidad, seguridad o autorización, se podrá presentar un acta de observación o una fuente institucional verificable.

9. Parte II: construcción del grafo

El equipo deberá transformar la problemática en un modelo de grafo.

9.1. Definición formal

El grafo deberá expresarse como:

$$G = (V, E),$$

donde:

$$V = \{v_1, v_2, \dots, v_n\}$$

representa el conjunto de vértices y

$$E = \{e_1, e_2, \dots, e_m\}$$

representa el conjunto de aristas.

Cuando el grafo sea ponderado, deberá definirse una función:

$$w : E \longrightarrow \mathbb{R}_{\geq 0},$$

donde $w(e)$ representa el peso de cada arista.

9.2. Requerimientos del modelo

El informe deberá especificar:

1. Qué representa cada vértice.
2. Qué representa cada arista.
3. Qué representa el peso de las aristas.
4. Si el grafo es dirigido o no dirigido.
5. Si el grafo es ponderado o no ponderado.
6. Las unidades utilizadas: metros, kilómetros, minutos, bolivianos u otras.
7. Las restricciones y supuestos adoptados.
8. La cantidad total de vértices y aristas.

El grafo utilizado deberá tener, salvo justificación técnica, al menos:

$$|V| \geq 10 \quad \text{y} \quad |E| \geq 15.$$

9.3. Representación obligatoria

El equipo deberá presentar:

- Representación gráfica del grafo.
- Lista de vértices.
- Lista de aristas.
- Matriz de adyacencia o matriz de pesos.
- Lista de adyacencia generada por el sistema.

10. Parte III: aplicación de algoritmos

10.1. Búsqueda en profundidad: DFS

El sistema deberá implementar el algoritmo de búsqueda en profundidad.

Deberá permitir seleccionar un nodo inicial y mostrar:

- Orden en que se visitan los nodos.
- Aristas utilizadas durante el recorrido.
- Nodos alcanzables desde el origen.
- Identificación de componentes conexos, cuando corresponda.
- Detección de ciclos o caminos alternativos, si resulta pertinente.

El equipo deberá explicar qué utilidad tiene DFS dentro de la problemática seleccionada.

10.2. Búsqueda en amplitud: BFS

El sistema deberá implementar el algoritmo de búsqueda en amplitud.

Deberá permitir:

- Seleccionar un nodo inicial.
- Mostrar el recorrido por niveles.
- Determinar el número mínimo de aristas entre dos nodos.
- Reconstruir el camino encontrado.
- Mostrar los nodos visitados y la estructura de la cola.

El equipo deberá explicar por qué BFS es adecuado para encontrar caminos mínimos cuando todas las conexiones tienen el mismo costo.

10.3. Algoritmo de Dijkstra

El sistema deberá implementar el algoritmo de Dijkstra para obtener el camino de menor costo entre un nodo origen y los demás nodos.

Deberá mostrar:

- Nodo origen y nodo destino.
- Distancia o costo mínimo.
- Secuencia completa del camino.
- Tabla de distancias calculadas.
- Nodo predecesor de cada vértice.
- Visualización del camino óptimo dentro del grafo.

Los pesos utilizados deberán ser mayores o iguales a cero:

$$w(u, v) \geq 0.$$

Si existen pesos negativos, el sistema deberá mostrar una advertencia indicando que Dijkstra no es aplicable.

10.4. Algoritmo de Kruskal

El sistema deberá generar un árbol de expansión mínima mediante Kruskal.

Deberá mostrar:

- Aristas ordenadas de menor a mayor peso.
- Aristas aceptadas.
- Aristas rechazadas por generar ciclos.
- Árbol de expansión mínima resultante.
- Costo total de la solución.

El sistema deberá verificar que el grafo sea no dirigido, ponderado y conexo antes de ejecutar el algoritmo.

10.5. Algoritmo de Prim

El sistema deberá implementar Prim permitiendo seleccionar un nodo inicial.

Deberá mostrar:

- Nodo inicial.
- Arista seleccionada en cada iteración.

- Conjunto de nodos incorporados.
- Árbol de expansión mínima.
- Costo total obtenido.

10.6. Comparación entre Kruskal y Prim

El equipo deberá comparar los resultados mediante una tabla como la siguiente:

Criterio	Kruskal	Prim
Costo total		
Número de aristas		
Orden de selección		
Tiempo de ejecución		
Ventajas observadas		
Limitaciones observadas		

Si existen varias soluciones óptimas, los árboles obtenidos pueden ser diferentes, pero deberán cumplir:

$$\text{Costo de Kruskal} = \text{Costo de Prim.}$$

Si los costos son diferentes, el equipo deberá revisar la implementación o justificar técnicamente el resultado.

11. Parte IV: desarrollo del sistema funcional

11.1. Nombre sugerido

Sistema Inteligente de Análisis y Optimización de Redes

El equipo podrá utilizar otro nombre relacionado con la problemática seleccionada.

11.2. Tipo de aplicación

El sistema podrá implementarse como:

- Aplicación web.

- Aplicación de escritorio.
- Aplicación móvil.
- Sistema multiplataforma.

No se considerará sistema funcional a un programa que únicamente muestre resultados fijos en la consola. Debe existir interacción con el usuario y posibilidad de modificar los datos.

11.3. Tecnologías sugeridas

El estudiante podrá utilizar el lenguaje y las herramientas de su preferencia. Algunas alternativas son:

- Python con Streamlit, Flask, Django o Tkinter.
- Java con JavaFX, Swing o Spring Boot.
- JavaScript o TypeScript con React, Vue, Angular o Node.js.
- C# con .NET.
- PHP con Laravel.
- Kotlin o Flutter para aplicaciones móviles.

Para la representación gráfica pueden utilizarse bibliotecas como NetworkX, Graphviz, Cytoscape.js, vis.js, D3.js u otras equivalentes.

12. Funcionalidades mínimas

El sistema deberá cumplir obligatoriamente con las siguientes funcionalidades:

1. Registrar, modificar y eliminar vértices.
2. Registrar, modificar y eliminar aristas.
3. Asignar pesos a las aristas.
4. Cargar datos desde un archivo CSV o JSON.
5. Validar datos incompletos, duplicados o incorrectos.
6. Mostrar gráficamente la red.
7. Generar la matriz o lista de adyacencia.

8. Ejecutar DFS desde un nodo seleccionado.
9. Ejecutar BFS desde un nodo seleccionado.
10. Ejecutar Dijkstra entre un origen y un destino.
11. Ejecutar Kruskal.
12. Ejecutar Prim desde un nodo seleccionado.
13. Resaltar visualmente los caminos y árboles obtenidos.
14. Mostrar los costos y resultados numéricos.
15. Comparar los resultados de Kruskal y Prim.
16. Generar un reporte con resultados y recomendaciones.
17. Permitir reiniciar el análisis o cargar un nuevo grafo.

13. Entradas y salidas del sistema

13.1. Entradas mínimas

- Nombre o código de cada vértice.
- Descripción de cada vértice.
- Nodo de origen.
- Nodo de destino.
- Aristas existentes.
- Peso de cada arista.
- Tipo de grafo.
- Algoritmo que se desea ejecutar.

13.2. Salidas mínimas

- Grafo completo.
- Matriz o lista de adyacencia.
- Recorrido DFS.
- Recorrido BFS.

- Camino mínimo calculado por Dijkstra.
- Distancia o costo mínimo.
- Árbol de expansión mínima de Kruskal.
- Árbol de expansión mínima de Prim.
- Costo total de cada árbol.
- Comparación de resultados.
- Recomendaciones relacionadas con el problema real.

14. Casos de prueba obligatorios

El equipo deberá probar el sistema, como mínimo, con los siguientes casos:

1. Grafo real investigado por el equipo.
2. Grafo conexo con diferentes pesos.
3. Grafo con un nodo aislado.
4. Grafo no conexo.
5. Grafo con más de un camino entre origen y destino.
6. Grafo con pesos repetidos.
7. Intento de ingresar una arista duplicada.
8. Intento de ejecutar Dijkstra con un peso negativo.
9. Comparación de Kruskal y Prim sobre el mismo grafo.
10. Modificación de una arista para comprobar cómo cambia la solución.

Para cada caso deberá registrarse:

- Datos de entrada.
- Resultado esperado.
- Resultado obtenido.
- Estado de la prueba: aprobada o no aprobada.
- Captura de pantalla.

15. Validación manual

No será suficiente mostrar únicamente los resultados producidos por el sistema.

El equipo deberá seleccionar un grafo reducido de entre cinco y siete nodos y resolver manualmente:

1. Un recorrido DFS.
2. Un recorrido BFS.
3. Un camino mínimo mediante Dijkstra.
4. Un árbol de expansión mínima mediante Kruskal.
5. Un árbol de expansión mínima mediante Prim.

Posteriormente, deberá comparar los resultados manuales con los resultados generados por el sistema.

16. Evidencias del proceso de desarrollo

Para demostrar que el proyecto fue desarrollado y comprendido por los estudiantes, se deberán presentar las siguientes evidencias:

1. Planteamiento inicial del problema.
2. Evidencia de recopilación de datos.
3. Primer boceto del grafo.
4. Diagrama o prototipo inicial de la interfaz.
5. Historial de avances del código fuente.
6. Repositorio Git con contribuciones identificables.
7. Registro de errores encontrados y correcciones realizadas.
8. Casos de prueba.
9. Capturas del sistema en distintas etapas.
10. Video breve de demostración.
11. Declaración de herramientas utilizadas.

Se recomienda realizar al menos cuatro entregas parciales:

Avance	Momento	Evidencia
1	Diagnóstico	Problema, usuarios, datos y evidencias del contexto.
2	Modelado	Grafo, matriz o lista de adyacencia y justificación de pesos.
3	Algoritmos	Implementación y pruebas de DFS, BFS, Dijkstra, Kruskal y Prim.
4	Sistema final	Aplicación funcional, validación, informe y defensa.

17. Uso responsable de inteligencia artificial

Se permite utilizar inteligencia artificial como herramienta de apoyo para:

- Aclarar conceptos.
- Revisar redacción.
- Identificar posibles errores.
- Proponer casos de prueba.
- Consultar sintaxis de programación.

Sin embargo, el equipo deberá cumplir las siguientes condiciones:

1. Declarar qué herramientas de inteligencia artificial utilizó.
2. Adjuntar los principales mensajes o instrucciones empleados.
3. Identificar qué partes fueron sugeridas por la herramienta.
4. Revisar, comprender y validar todo código incorporado.
5. Explicar individualmente cualquier parte del sistema.
6. Presentar datos y evidencias obtenidos del contexto real.

No se admitirán como trabajo válido:

- Datos ficticios presentados como reales.

- Entrevistas, fotografías o evidencias inventadas.
- Código que los integrantes no puedan explicar.
- Resultados copiados sin validación.
- Aplicaciones generadas automáticamente sin adaptación al problema.

18. Estructura sugerida del proyecto

```
proyecto_grafos/  
|  
|-- app/  
|   |-- main.py  
|   |-- algoritmos.py  
|   |-- grafo.py  
|   |-- validaciones.py  
|   '-- reportes.py  
|  
|-- datos/  
|   |-- vertices.csv  
|   '-- aristas.csv  
|  
|-- pruebas/  
|   '-- test_algoritmos.py  
|  
|-- evidencias/  
|   |-- entrevistas/  
|   |-- fotografias/  
|   '-- capturas/  
|  
|-- documentacion/  
|   |-- informe.pdf  
|   '-- manual_usuario.pdf  
|  
|-- requirements.txt  
'-- README.md
```

La estructura podrá modificarse según la tecnología utilizada.

19. Formato sugerido de los datos

19.1. Archivo de vértices

id,nombre,descripcion

V01,Almacen central,Punto de salida de productos

V02,Sucursal norte,Punto de entrega

V03,Sucursal sur,Punto de entrega

19.2. Archivo de aristas

origen,destino,peso,unidad

V01,V02,8.5,kilometros

V01,V03,12.2,kilometros

V02,V03,6.4,kilometros

20. Producto final esperado

El equipo deberá entregar:

1. Informe académico en formato PDF.
2. Descripción y diagnóstico del problema.
3. Evidencias de la investigación realizada.
4. Base de datos original.
5. Construcción formal del grafo.
6. Matriz y lista de adyacencia.
7. Desarrollo manual de los algoritmos solicitados.
8. Código fuente completo.
9. Sistema funcional.
10. Repositorio Git o historial verificable de desarrollo.
11. Casos de prueba.
12. Capturas de pantalla.
13. Manual de usuario.
14. Video de demostración de entre cinco y ocho minutos.
15. Conclusiones y recomendaciones.
16. Declaración de uso de inteligencia artificial.

21. Estructura del informe

El informe deberá contener:

1. Portada.
2. Índice.
3. Introducción.
4. Descripción de la problemática.
5. Objetivos.
6. Metodología de recopilación de datos.
7. Evidencias del contexto real.
8. Fundamento teórico.
9. Construcción y representación del grafo.
10. Aplicación de DFS.
11. Aplicación de BFS.
12. Aplicación de Dijkstra.
13. Aplicación de Kruskal.
14. Aplicación de Prim.
15. Comparación de resultados.
16. Descripción del sistema.
17. Casos de prueba.
18. Interpretación de resultados.
19. Recomendaciones.
20. Conclusiones.
21. Referencias bibliográficas.
22. Anexos.

22. Defensa y demostración

Durante la defensa, el docente podrá solicitar:

1. Modificar o agregar un vértice.
2. Modificar el peso de una arista.
3. Cambiar los nodos de origen y destino.
4. Ejecutar nuevamente un algoritmo.
5. Explicar una sección específica del código.
6. Predecir el resultado antes de ejecutar el sistema.
7. Comparar BFS con Dijkstra.
8. Explicar por qué una arista fue rechazada por Kruskal.
9. Cambiar el nodo inicial de Prim.
10. Corregir un error sencillo durante la demostración.

La calificación individual podrá modificarse si un integrante no demuestra conocimiento suficiente del sistema entregado.

23. Preguntas orientadoras para el análisis

El informe deberá responder, como mínimo, las siguientes preguntas:

1. ¿Por qué la problemática puede representarse mediante un grafo?
2. ¿Qué representan los vértices y las aristas?
3. ¿Cómo se determinaron los pesos?
4. ¿Qué información proporciona DFS?
5. ¿Qué diferencia práctica existe entre DFS y BFS?
6. ¿En qué situación BFS produce un camino mínimo?
7. ¿Por qué Dijkstra requiere pesos no negativos?
8. ¿Cuál es la ruta óptima encontrada?
9. ¿Qué ahorro o mejora produce la ruta?

10. ¿Cuál es el costo del árbol de expansión mínima?
11. ¿Kruskal y Prim generaron el mismo árbol?
12. ¿Por qué pueden obtener árboles distintos con el mismo costo?
13. ¿Cómo cambia la solución al modificar una conexión?
14. ¿Qué limitaciones tiene el modelo?
15. ¿Qué decisión real podría tomarse con los resultados?

24. Conclusión esperada

Las conclusiones no deberán limitarse a indicar que los algoritmos funcionaron. El equipo deberá explicar:

- Qué se descubrió sobre la problemática.
- Qué alternativa resultó más conveniente.
- Qué costos, distancias o tiempos podrían reducirse.
- Qué usuarios se beneficiarían.
- Qué limitaciones presentan los datos.
- Qué mejoras podrían incorporarse en una futura versión.

25. Criterios de aprobación

Para aprobar la práctica será indispensable:

1. Presentar una problemática real y evidencias verificables.
2. Implementar los cinco algoritmos solicitados.
3. Entregar un sistema funcional.
4. Realizar correctamente la validación manual.
5. Presentar el código fuente.
6. Defender individualmente el trabajo.
7. Obtener al menos el puntaje mínimo definido por la institución.

La ausencia del sistema funcional, del código fuente o de las evidencias del problema real impedirá alcanzar la calificación máxima.

26. Referencias bibliográficas sugeridas

- Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. *Introduction to Algorithms*. MIT Press.
- Rosen, K. H. *Discrete Mathematics and Its Applications*. McGraw-Hill.
- Sedgewick, R. y Wayne, K. *Algorithms*. Addison-Wesley.
- West, D. B. *Introduction to Graph Theory*. Prentice Hall.